

Labo 04 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — `COPY ../commun/...`

Réponse. Le build n'a accès qu'au **contexte** — le dossier passé en argument (`.`, donc `~/projets/api/`). `../commun` est en dehors : Buildah ramène le chemin à l'intérieur du contexte (`possible escaping context directory`), n'y trouve rien, et échoue. `-f` ne désigne que le Dockerfile, pas le périmètre ; un chemin absolu est lui aussi ramené dans le contexte ; `sudo` ne change rien à un problème de périmètre, pas de droits. Solution : construire depuis le dossier parent (`podman build -f api/Dockerfile -t api:1.0 ~/projets`) avec des chemins `COPY api/...commun/...`, ou copier `config.yml` dans le projet avant le build — ou mieux, ne pas l'embarquer du tout et l'injecter à l'exécution (labo 08).

Pourquoi. Le contexte est une frontière de sécurité et de reproductibilité : un Dockerfile ne peut dépendre que de ce qu'on lui donne explicitement. Avec Docker, c'est physique (le contexte est archivé et envoyé au daemon) ; avec Podman, c'est une règle appliquée par Buildah — même résultat.

Nuance. Podman accepte plusieurs contextes nommés : `podman build --build-context commun=../commun .` puis `COPY --from=commun config.yml /app/`. C'est la solution propre quand un fichier partagé doit vraiment entrer dans plusieurs images.

Exemple.

```
podman build -f Dockerfile.hors-contexte -t essai .
# Error: building at STEP "COPY ../commun/secret.txt /": ... possible escaping context directory
error
```

Question 2 — `transferring context`, puis plus rien

Réponse. Sous Docker, le client empaquette **tout** le dossier (1,1 Go) et l'envoie au daemon avant la première instruction : c'est le `transferring context`. Sous Podman, Buildah lit le dossier sur place, sans archive ni transfert : la lenteur disparaît. Mais le second risque est intact : un `COPY . .` embarque `node_modules` et `.git` **dans l'image** — 1,1 Go inutiles, plus l'historique Git complet (et ses éventuels secrets) offerts à quiconque télécharge l'image. `.dockerignore` reste donc obligatoire ; il ne sert plus la vitesse, il sert le contenu.

Pourquoi. Le contexte a deux rôles : ce qui est *envoyé* (Docker seulement) et ce qui est *copiable*. Podman supprime le premier coût, pas le second.

Nuance. `.git` dans une image est une fuite fréquente et grave : l'historique contient souvent des identifiants supprimés « depuis ». Et sans `.dockerignore`, un fichier modifié dans `node_modules` invalide aussi le cache du `COPY`.

Exemple.

```
podman build -q -f Dockerfile.tout -t t . # image de 218 Mo avec node_modules
printf 'node_modules\n.git\n' > .dockerignore
podman build -q -f Dockerfile.tout -t t2 . # 8,7 Mo
```

Question 3 — EXPOSE et le port qui ne répond pas

Réponse. Non. EXPOSE est une **déclaration** : elle documente que l'application écoute sur 8080 et alimente podman ps et -P. Elle ne crée aucune redirection depuis l'hôte. Sans -p 8080:8080, le port n'est accessible que depuis le réseau des conteneurs.

Pourquoi. Publier un port est une décision de déploiement (quel port hôte, quelle interface), pas une propriété de l'image. L'image dit « j'écoute sur 8080 » ; l'exploitant décide « je l'expose en 18080 ».

Nuance. -P (majuscule) publie automatiquement tous les ports EXPOSE sur des ports aléatoires de l'hôte : c'est là que la déclaration devient utile. Et en rootless, -p 80:8080 échoue (port privilégié) ; choisissez ≥ 1024 ou réglez net.ipv4.ip_unprivileged_port_start.

Exemple.

```
podman run -d --name a mon-api:1.0 && curl -m 2 localhost:8080      # échec
podman run -d --name b -p 8080:8080 mon-api:1.0 && curl localhost:8080/actuator/health #
{"status":"UP"}
podman port b                                                       # 8080/tcp -> 0.0.0.0:8080
```

Question 4 — RUN java contre CMD java

Réponse. A lance l'API **pendant le build** : RUN exécute la commande au moment de la construction, l'API démarre, ne se termine jamais... et le build reste bloqué (ou, si l'API se termine, l'image ne contient qu'une couche inutile). B est correct : CMD enregistre la commande à lancer au podman run.

Pourquoi. RUN sert à préparer le système de fichiers (installer, compiler, copier) ; CMD / ENTRYPOINT décrivent le processus principal du futur conteneur. Confondre les deux, c'est confondre construction et exécution.

Nuance. B serait encore meilleur avec ENTRYPOINT + CMD (question 5) et un USER. Et un RUN java -jar a un usage légitime : lancer une **tâche finie** au build, comme java -Djarmode=tools -jar app.jar extract (labo 05).

Exemple.

```
podman build -f A -t a .      # STEP 3/3: RUN java -jar ... - n'aboutit jamais
podman build -f B -t b . && podman run -d -p 18080:8080 b
```

Question 5 — ENTRYPOINT + CMD contre CMD seul

Réponse. A : podman run img → java -jar /app/api.jar --spring.profiles.active=prod ; podman run img --debug → java -jar /app/api.jar --debug (le CMD est remplacé, l'ENTRYPOINT reste). B : podman run img → la même commande complète ; podman run img --debug → exécute --debug **tout seul**, sans java : erreur executable file not found. Seule B permet podman run img sh (le CMD entier est remplacé par sh). Avec A, podman run img sh lance java -jar api.jar sh ; il faut podman run --entrypoint sh img.

Pourquoi. Les arguments de run remplacent le CMD et s'ajoutent à l'ENTRYPOINT. A est fait pour une image « application », B pour une image « outil ».

Nuance. En forme `exec`, A et B mettent `java` en PID 1. Une variante courante en entreprise : `ENTRYPOINT ["java", "-jar", "app.jar"]` sans `CMD`, et la configuration par variables d'environnement — les arguments ne servent qu'au débogage.

Exemple.

```
timeout 5 podman run --rm api-labo:1.0 --debug | head -1      # Arguments recus : --debug
podman run --rm --entrypoint sh api-labo:1.0 -c 'echo ok'     # ok
```

Question 6 — Dix secondes, `resorting to SIGKILL`, pas de hooks

Réponse. `CMD java -jar /app/api.jar` est une forme **shell** : le moteur exécute `/bin/sh -c "java -jar /app/api.jar"`. Sur une base Debian/Ubuntu, `/bin/sh` est `dash`, qui lance Java comme enfant et reste PID 1. `podman stop` envoie `SIGTERM` au PID 1 — le shell — qui ne le transmet pas. Java ne reçoit rien, ses *shutdown hooks* ne s'exécutent pas ; après 10 secondes, Podman annonce `resorting to SIGKILL` et tue tout (137). Correction : `CMD ["java", "-jar", "/app/api.jar"]`. Sur Alpine, `/bin/sh` est `ash` (busybox), qui **s'auto-remplace** par la commande quand elle est simple : Java devient PID 1, reçoit `SIGTERM`, et le problème est invisible en test.

Pourquoi. Un shell POSIX n'a aucune obligation de relayer les signaux à ses enfants ; `dash` ne le fait pas. La forme `exec` supprime le shell, donc la question.

Nuance. Même Alpine ne sauve pas un `CMD` shell avec `&&`, `|` ou une variable : le shell doit alors rester. La règle « forme `exec` toujours » évite d'avoir à connaître le comportement de chaque shell.

Exemple.

```
podman exec s-deb ps -o pid,args | head -3      # 1 /bin/sh -c java ... 2 java ...
time podman stop s-deb                          # resorting to SIGKILL, 10 s, code 137
```

Question 7 — `$JAVA_OPTS` non interprété

Réponse. En forme `exec`, il n'y a **pas de shell** : `$JAVA_OPTS` est passé à Java tel quel, comme une chaîne de six caractères. Deux corrections : (1) forme shell explicite avec `exec` — `ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/api.jar"]` : coût, une dépendance à `sh` et une ligne moins lisible ; (2) supprimer la variable — Java lit lui-même `JAVA_TOOL_OPTIONS` dans l'environnement, donc `ENV JAVA_TOOL_OPTIONS="-Xmx512m"` et `ENTRYPOINT ["java", "-jar", "/app/api.jar"]` : coût, un message `Picked up JAVA_TOOL_OPTIONS` sur `stderr` au démarrage, et une variable qui s'applique à *tous* les processus Java du conteneur.

Pourquoi. L'expansion des variables est un service du shell. La forme `exec` est un tableau d'arguments transmis directement à l'appel système `execve`.

Nuance. La forme (1) sans `exec` recréerait le problème de la question 6. Et pour la mémoire spécifiquement, `-XX:MaxRAMPercentage=75` vaut mieux qu'un `-Xmx` fixe : la JVM s'adapte au cgroup (labo 10).

Exemple.

```
podman run --rm -e JAVA_TOOL_OPTIONS="-Xmx256m" api-labo:1.0 --debug 2>&1 | head -1
# Picked up JAVA_TOOL_OPTIONS: -Xmx256m
```

Question 8 — `--build-arg` et le secret

Réponse. Non. La valeur d'un `ARG` n'est pas dans `Config.Env`, mais elle est enregistrée dans l'**historique** de chaque instruction qui l'utilise, et dans le cache de build. `podman history --no-trunc api:1.0` l'affiche en clair (`|1 DB_PASSWORD=Secr3t! /bin/sh -c ...`). Quiconque a l'image a le mot de passe.

Pourquoi. L'historique décrit comment chaque couche a été produite, arguments compris — c'est ce qui permet le cache. Un `ARG` est une entrée du build, donc du cache, donc de l'historique.

Nuance. La bonne pratique : le secret n'a rien à faire au *build*. S'il est indispensable (dépôt Maven privé), `RUN --mount=type=secret,id=settings ...` le rend disponible pendant une instruction sans jamais l'écrire dans une couche (labo 08). Et un multi-stage ne protège que si le secret n'est utilisé que dans un stage jeté (labo 05).

Exemple.

```
podman history --no-trunc api-labo:arg --format '{{.CreatedBy}}' | grep DB_PASSWORD
# |1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build avec $DB_PASSWORD" > /trace.txt
```

Question 9 — Six minutes par build Maven

Réponse.

```
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline      # téléchargement des dépendances, mis en cache
COPY src ./src
RUN mvn -q package -DskipTests      # compilation seule
```

Le cache réutilise une instruction si son texte **et** ses entrées sont inchangés. `pom.xml` change rarement : la couche `dependency:go-offline` (les cinq minutes de téléchargement) reste `Using cache`. Seule la compilation est rejouée quand un `.java` change. Le build restera lent quand `pom.xml` change — ajout ou montée de version d'une dépendance — puisque la couche des dépendances est alors invalidée.

Pourquoi. `COPY . /app` place *tout* le code avant Maven ; tout commit invalide la copie, donc tout ce qui suit.

Nuance. `dependency:go-offline` n'est pas parfait (certains plugins téléchargent encore au `package`). Un `RUN --mount=type=cache,target=/root/.m2` (labo 05) conserve le dépôt local entre builds, même quand `pom.xml` change. Et avec Podman, aucun de ces gains ne dépend d'un daemon : le cache est dans votre stockage utilisateur.

Exemple.

```
time podman build -t api-labo:2.1 .      # RUN ... sleep 5 : --> Using cache ; 0,7 s
```

Question 10 — Trois `RUN apt`

Réponse. (1) **Trois couches au lieu d'une** : les listes apt (`/var/lib/apt/lists`, ~40 Mo) sont écrites dans la couche 2 ; la couche 3 ne fait que les masquer, l'image garde les 40 Mo. (2) `apt-get update` **isolé** est mis en cache : dans des semaines, une modification de la ligne `install`

réutilisera des index périmés (paquets introuvables, versions anciennes). (3) **Pas de** `--no-install-recommends` et `vim` dans une image de production : des dizaines de Mo de paquets inutiles, autant de surface d'attaque. Version correcte :

```
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Pourquoi. Une couche est immuable ; le nettoyage n'a d'effet que dans la couche qui a créé les fichiers. Et le cache travaille instruction par instruction : `update` et `install` doivent être solidaires.

Nuance. Sur Alpine : `apk add --no-cache curl` fait tout en une ligne. Et `vim` dans une image n'est jamais justifié : `podman exec` avec un éditeur temporaire, ou pas d'éditeur du tout (image distroless, labo 05).

Exemple.

```
podman history img --format 'table {{.Size}}\t{{.CreatedBy}}' # la couche "rm -rf" fait 0B,
celle du dessus 40 Mo
```

Question 11 — `Using cache` puis tout reconstruit

Réponse. La règle : **une instruction invalidée invalide toutes celles qui la suivent**, et le cache se lit de haut en bas. Jour 1 : seul le dernier `COPY` a changé, tout ce qui précède est identique → huit `Using cache`, puis reconstruction des deux dernières. Jour 2 : une instruction insérée en troisième position change le texte du Dockerfile à partir de la ligne 3 → les étapes 3 à 10 sont nouvelles, donc reconstruites — même si leur contenu n'a pas bougé.

Pourquoi. La clé de cache d'une étape est (couche parente, instruction, entrées). Changer la couche parente change la clé de tout ce qui suit.

Nuance. C'est pour cela que les `ENV`, `ARG` et `LABEL` variables (numéro de build, date) se placent **en fin** de Dockerfile, et que les métadonnées stables se placent tôt. Un `ARG BUILD_DATE` en ligne 2 invalide tout, à chaque build.

Exemple.

```
podman build -t api-labo:3.1 . # STEP 3/5: COPY ... (rejoué) puis STEP 4/5: RUN ... sleep 5
(rejoué aussi)
```

Question 12 — `ADD` contre `COPY`

Réponse. Deux comportements propres à `ADD` : il **décompresse** automatiquement une archive locale (`.tar`, `.tar.gz`, `.tar.xz`) vers la destination, et il **télécharge** une URL. La recommandation officielle est `COPY` parce que ces comportements sont implicites : un `ADD fichier.tar.gz /app/` qui décompresse alors qu'on voulait copier l'archive, une URL téléchargée sans vérification ni cache et sans `rm` possible dans la même couche. Le seul cas justifié : extraire une archive **locale** en une instruction (`ADD rootfs.tar.gz /`).

Pourquoi. Un Dockerfile doit être lisible sans surprise ; `COPY` fait une chose. Pour une URL, `RUN curl ... && tar ... && rm ...` dans un seul `RUN` est explicite et nettoyable.

Nuance. Les deux acceptent `--chown` (et `--chmod`), utiles avant un `USER`. Et `COPY --from=` (labo 05) n'a pas d'équivalent `ADD`.

Exemple.

```
ADD app.tar.gz /opt/          # /opt/app/... décompressé
COPY app.tar.gz /opt/         # /opt/app.tar.gz tel quel
```

Question 13 — `USER` trop tôt, et `HUSER 100999`

Réponse. `USER` s'applique à toutes les instructions suivantes, `RUN` compris. Placé après `FROM`, il fait exécuter `apt-get install` et `mkdir` par l'UID 1000, qui n'a pas le droit d'écrire dans `/usr`, `/var` ou `/` : `Permission denied`. Dans un Dockerfile bien écrit, `USER` se place **juste avant** `ENTRYPOINT` / `CMD`, après avoir installé, créé les dossiers et ajusté leur propriétaire (`chown`, `COPY -chown`). En rootless, l'UID 1000 du conteneur est projeté sur l'hôte via `/etc/subuid` : le premier UID « supplémentaire » (1) correspond à 100000, donc 1000 → 100999. `USER` reste utile : (a) il retire à l'application les droits root *dans* le conteneur (modifier les fichiers de l'image, écouter sur 80, installer des paquets) ; (b) la même image tournera sous Docker ou Kubernetes, où root est vraiment root ; (c) les scanners de sécurité et les politiques d'admission refusent les images sans `USER`.

Pourquoi. Le namespace `user` protège l'hôte ; `USER` protège le *conteneur* et ce qu'il contient. Les deux couches sont complémentaires.

Nuance. `USER 1000:1000` sans créer l'utilisateur fonctionne (Podman ajoute même une entrée `/etc/passwd` à la volée), mais certains programmes veulent un `HOME` ou un nom : `RUN adduser -D -u 1000 app` puis `USER app` est plus robuste.

Exemple.

```
podman top u user,huser          # 1000 100999
podman run --rm --entrypoint sh api-labo:user-ok -c 'touch /app/x'  # Permission denied :
bien.
```

Question 14 — « Un seul `RUN` »

Réponse. Il a raison quand des fichiers créés par une instruction sont supprimés par une autre (installation + nettoyage, décompression + suppression de l'archive) : séparés, les fichiers restent dans l'image. Il a tort quand le regroupement mélange du stable et du volatil : un `RUN` unique qui télécharge les dépendances **et** compile le code est invalidé à chaque commit, donc retélécharge tout ; et une couche unique de 300 Mo est retransférée entièrement à chaque `push`, alors que cinq couches dont quatre stables ne transfèrent que le delta.

Pourquoi. Le nombre de couches n'a en soi presque aucun coût. Ce qui compte, c'est **quels fichiers vivent dans quelle couche** (taille) et **à quelle fréquence chaque couche change** (cache et transfert).

Nuance. Règle pratique : un `RUN` par « unité de changement » — installation système (stable), dépendances (semi-stable), code (volatil). Le multi-stage (labo 05) et le découpage du JAR Spring Boot appliquent exactement cette logique.

Exemple.

```
podman history mon-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # une couche par unité  
de changement
```